# Banking Management System

Blueprint, C structure diagram, code walkthrough, and viva preparation

## What this project demonstrates

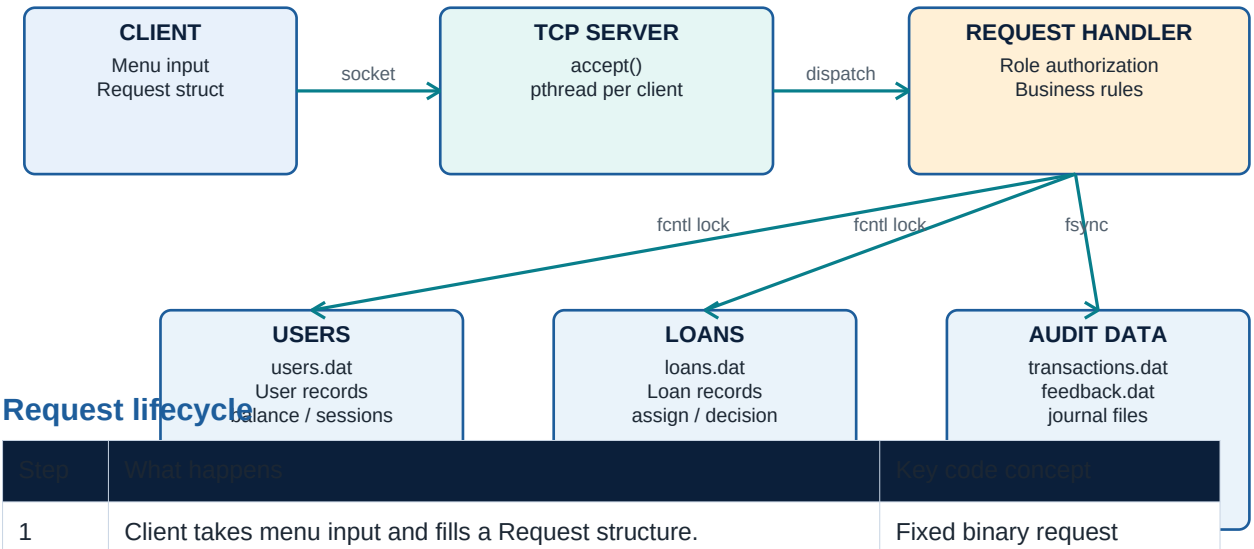
This is a client-server banking simulation written in C for Linux. A TCP server owns the persistent database files. Clients authenticate by role and send fixed-size **Request** records; the server returns **Response** records. The design intentionally demonstrates sockets, POSIX threads, file locking, semaphores, persistence, recovery journals, and role-based access control.

| Viva topic | Where it appears in the project | One sentence to say |
|---|---|---|
| Socket programming | bank_server accepts TCP clients; bank_client connects to it | The server centralizes the database and lets many clients connect concurrently. |
| Threads | pthread_create creates one detached worker per client | Each client is handled independently without blocking new connections. |
| File management | Binary .dat files store users, loans, transactions and feedback | Records remain after the program stops, unlike in-memory arrays. |
| Synchronization | fcntl locks plus a named semaphore | Locks prevent race conditions while a balance or record is updated. |
| Security | salted SHA-256 password digest and one active session | The password is verified as a digest and duplicate login is rejected. |

### Suggested demonstration order

- Customer applies for a loan.
- Manager lists pending loan IDs and assigns one to an employee.
- Employee approves it; the amount is credited and a passbook entry is created.
- Customer checks loan status, balance, and transaction history.

# 1. System Blueprint



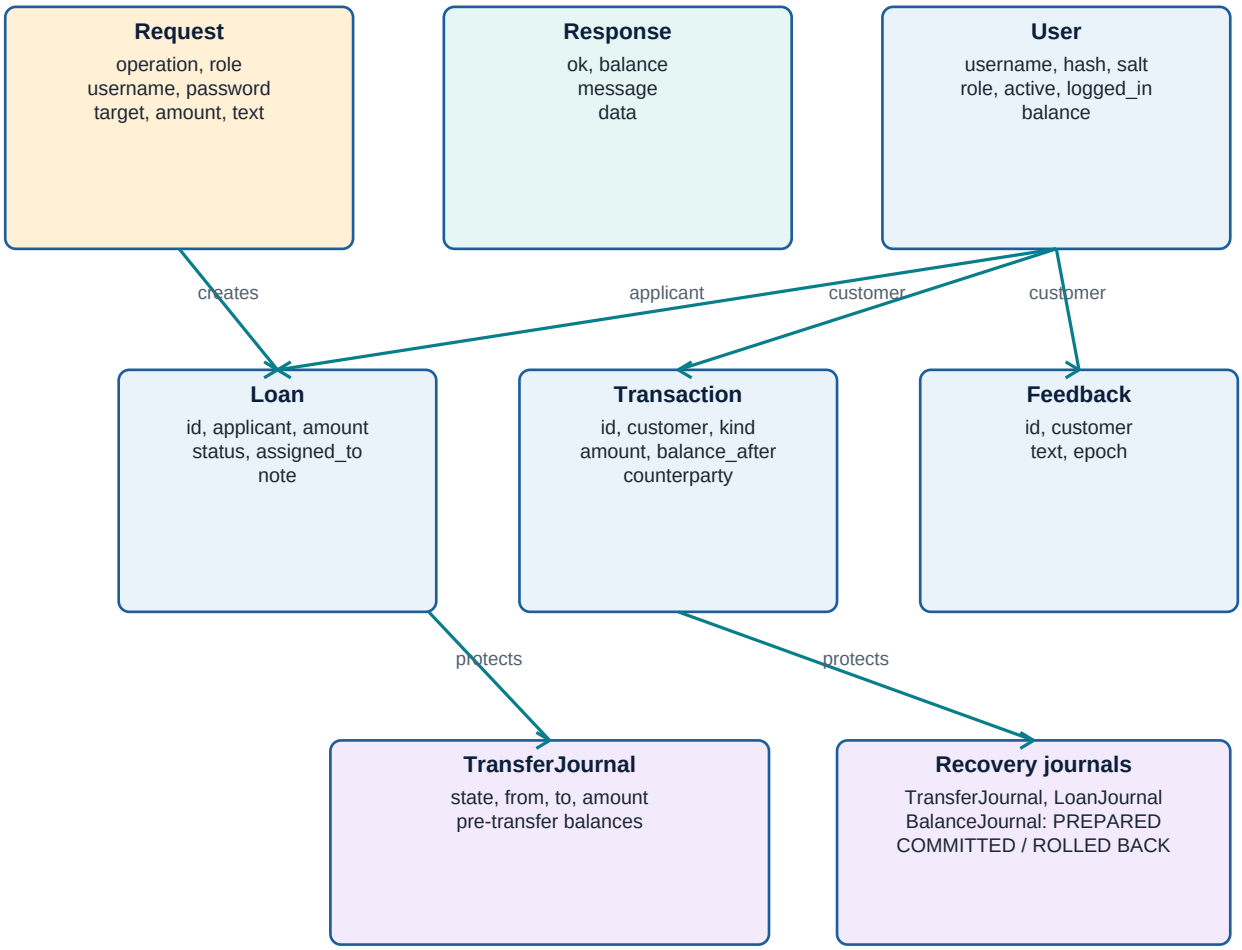

## Request lifecycle

| Step | What happens | Key code concept |
|---|---|---|
| 1 | Client takes menu input and fills a Request structure. | Fixed binary request protocol |
| 2 | Client sends Request over TCP; server receives it fully. | send()/recv() loops |
| 3 | A server thread checks the logged-in role and dispatches the operation. | Role-based authorization |
| 4 | The operation locks the relevant file, reads or changes a record, then fsyncs mutations. | fcntl advisory lock + durability |
| 5 | Server sends a Response containing success/failure, message, balance, and optional rows. | Request-response protocol |

## Files created in the data directory

| File | Purpose |
|---|---|
| users.dat | User credentials (salted digest), role, active/session state and customer balance. |
| loans.dat | Loan applications, employee assignment, status and decision note. |
| transactions.dat | Passbook-style deposits, withdrawals, transfers and approved-loan credits. |
| feedback.dat | Customer feedback visible to managers. |
| journal.log, transfer_journal.dat, loan_journal.dat, balance_journal.dat | Audit entries and recovery data for incomplete transfers, loan credits, deposits, and withdrawals. |

# 2. C Structure Diagram



## How to explain the diagram

- **Request** and **Response** are network messages. They keep the client and server protocol simple and predictable.
- **User** is the core record. For customers, its *balance* field is meaningful; for staff it remains zero.
- **Loan**, **Transaction**, and **Feedback** are appendable business records linked by username.
- TransferJournal, LoanJournal, and BalanceJournal capture the previous balance before a critical update. If a restart finds PREPARED state, recovery rolls back the unfinished change.

## Important enum values

| Enum | Meaning |
|---|---|
| Role | ROLE_CUSTOMER, ROLE_EMPLOYEE, ROLE_MANAGER, ROLE_ADMIN |
| Loan status | 0 PENDING, 1 APPROVED, 2 REJECTED |
| Journal state | 1 PREPARED, 2 COMMITTED, 3 ROLLED BACK |

# 3. Core Workflows

## Login and one-session rule

During login, the server locks **users.dat**, finds the username, computes a SHA-256 digest using the saved salt, verifies the selected role, checks that the account is active and not already logged in, then writes *logged_in = 1*. Logout or a disconnected client clears this flag.

## Transfer workflow

| Order | Action | Why it matters |
|---|---|---|
| 1 | Validate positive amount, recipient, active customer accounts and sufficient funds. | Rejects invalid business operations before changing data. |
| 2 | Acquire named semaphore and exclusive users.dat lock. | Avoids two simultaneous withdrawals or transfers reading the same old balance. |
| 3 | Write PREPARED TransferJournal with both previous balances and fsync it. | Provides a recovery point before account values change. |
| 4 | Debit sender and credit receiver under one users.dat lock, then fsync. | The paired account update is serialized. |
| 5 | Mark journal COMMITTED and append two transaction/passbook records. | Leaves audit evidence for both customers. |

## Loan workflow

| Actor | Action | Result |
|---|---|---|
| Customer | Applies for a loan with amount and purpose. | A PENDING Loan record receives an ID. |
| Manager | Views pending IDs and assigns a loan to an active employee. | Only that employee can decide it. |
| Employee | Approves or rejects an assigned PENDING loan. | Approval uses a LoanJournal, credits customer balance, commits decision, then adds LOAN_CREDIT to history. |
| Customer | Uses My loan status and Balance/History. | Sees the decision and credited money after approval. |

## Quick command reference

```
make clean && make
./bank_server data
./bank_client 127.0.0.1
```

# 4. Roles and Menus

| Role | Main permissions | Good viva explanation |
|---|---|---|
| Customer | Balance, deposit, withdraw, transfer, loan application/status, password, feedback, history. | A customer can only affect their own account and records. |
| Employee | Open/modify/close customer accounts, view assigned loans, approve/reject, view passbook. | Loan authorization is constrained by assigned_to, so an employee cannot decide another employee's loan. |
| Manager | Activate/deactivate customers, list pending loans, assign loans, review feedback. | The manager supervises allocation and account state but does not directly transfer money. |
| Administrator | Create customers, employees, and managers; modify roles/details; activate/deactivate customers, employees, and managers. | The administrator manages every non-admin account and can invalidate its current session. |

## Authorization pattern

The server never trusts a client menu by itself. After login, it stores the session username and role on the server-side connection. **handle_request()** selects operations by that role. Each sensitive operation also performs its own checks - for example, loan_decision verifies that the loan is still pending and is assigned to the current employee.

## Useful demo data

| Role | Username | Default password |
|---|---|---|
| Administrator | admin | admin123 |
| Manager | manager | manager123 |
| Employee | employee | employee123 |
| Customer | customer | customer123 |

## Account deactivation and forced logout

When an employee closes a customer account, or a manager/admin deactivates an account, the server sets *active = 0* and *logged_in = 0* in users.dat while the user record is locked. Before every authenticated request, *ensure_session_active()* rereads that record. Therefore the next request from the old socket is rejected and its server-side session is cleared. The customer client recognizes the message and returns to the login screen.

Important: change the demo passwords before a real demonstration and do not run multiple server instances against the same data folder.

# 5. Concurrency, ACID and Security

## Race condition example

Without locking, two customer threads can both read balance 1000, both decide that a withdrawal of 800 is allowed, and each write 200. The bank has allowed withdrawal of 1600. In this project, the named semaphore and exclusive file lock make the read-check-write sequence run one at a time for balance-changing operations.

| Property | How the project addresses it | Viva wording |
|---|---|---|
| Atomicity | Prepared journal stores prior values; failed or incomplete critical updates are rolled back. | A transfer or loan credit is treated as all-or-nothing. |
| Consistency | Validation checks account state, role, positive amount, sufficient balance, assigned pending loan. | Every operation preserves the banking rules. |
| Isolation | Named semaphore and fcntl locks serialize conflicting updates. | Concurrent clients cannot interleave a balance update. |
| Durability | fsync is called after mutation and journal transitions. | Acknowledged changes are flushed to disk. |

## Security points

- Passwords are stored as a salted SHA-256 digest, not printed back to the client.
- The client provides a role at login, but the server compares it with the stored role before opening a session.
- The logged_in flag enforces one session per user; cleanup runs when a client disconnects.
- All inputs are validated for required strings, IDs, role bounds, and safe monetary values before writes.
- Deactivation also clears logged_in; request-time session validation prevents a deactivated customer from continuing to use an old connection.

## Limitations to acknowledge honestly

This is an educational local-network simulation, not a production bank. It does not use TLS, a real relational database, multi-factor authentication, encrypted files, interest calculation, or a full crash-safe transaction database. The goal is to demonstrate Linux system-programming concepts in a manageable mini-project.

# 6. Viva Questions and Short Answers

**Why use a client-server model?**

It centralizes persistent bank data on the server. Many clients can use the same database without each client editing local files.

**Why use TCP instead of UDP?**

Bank requests need reliable, ordered delivery. TCP provides a connected byte stream, while the code still uses send_all and recv_all to handle partial transfers.

**Why are file locks required if we already use threads?**

pthread synchronization only covers threads in one process. fcntl file locks also coordinate access if another process opens the same data file.

**What is the role of the named semaphore?**

It serializes balance-changing operations so the read-check-write sequence cannot race.

**What happens if a customer tries to log in twice?**

The first login sets logged_in to 1 under a users.dat lock. The second login sees that flag and is rejected.

**How does transfer avoid partial update?**

A prepared journal records old balances before the debit/credit. Both records are changed while locked and fsynced; startup recovery rolls back an unfinished journal.

**How is an approved loan reflected?**

The employee's approval credits the applicant balance, changes the loan status to APPROVED, commits a loan journal, and writes a LOAN_CREDIT transaction.

**How is employee loan access controlled?**

The loan has assigned_to. The server rejects a decision unless assigned_to matches the logged-in employee and the loan is still pending.

**Why use fixed-size structs?**

They make binary file records and the socket protocol easy to read/write at known offsets. The tradeoff is client and server must use compatible struct layouts.

**How do concurrent managers avoid assigning the same loan twice?**

assign_loan locks the loans file and uses a pthread mutex inside the server. It checks that status is PENDING and assigned_to is empty, so the first manager writes the assignment and the other request fails.

**What happens when an account is deactivated while it is logged in?**

The update clears logged_in and active under a user-record lock. ensure_session_active rejects the next request from that connection, so the session cannot perform another banking operation.

**Why is BalanceJournal needed when there is journal.log?**

journal.log is audit text, while BalanceJournal stores structured recovery data including the old balance and state. Startup code can safely roll back a PREPARED deposit or withdrawal.

**What would you improve for production?**

TLS, a real DBMS with transactions, stronger password handling such as Argon2, audit authentication, authorization policies, input framing, backups, and monitoring.

## Last-minute viva checklist

- Be ready to explain one complete loan flow and one complete transfer flow.
- Mention both pthreads and fcntl locking, then distinguish their scope.
- Use the four ACID words with the concrete code mechanisms above.

- State one limitation clearly; examiners value a realistic answer.

# 7. Code Map and Viva Demonstration Script

## Where to point in the code

| File | What to explain in viva | Keywords to mention |
|---|---|---|
| bank.h | Shared protocol and persistent record layouts. | Request, Response, User, Loan, Transaction, journals |
| client.c | Role menus collect input and send a fixed-size Request. | customer_menu, employee_menu, manager_menu, admin_menu |
| server.c | Server accepts sockets, creates detached pthreads, dispatches by stored role, and owns all files. | client_thread, handle_request, login, ensure_session_active |
| sha256.c | Password-digest helper used with a unique salt. | SHA-256, password_digest |
| integration_test.sh | Automated end-to-end demonstration of the main workflow. | server, customer, manager, employee, administrator |

## Five-minute live demonstration

| Minute | Action | What to say |
|---|---|---|
| 1 | Start server, log in as customer, deposit and apply for a loan. | The server validates the request and persists fixed-size records. |
| 2 | Log in as manager, list pending loans, assign one to an employee. | The assignment is protected so two managers cannot claim the same unassigned loan. |
| 3 | Log in as employee, open assigned loans, approve the loan. | Approval credits the customer, commits the loan journal, and adds LOAN_CREDIT to the passbook. |
| 4 | Customer checks balance and history. | The credit is visible in both current balance and transactions.dat. |
| 5 | Deactivate the customer from staff/admin, then make one more customer request. | The account's active/logged_in flags cause ensure_session_active to terminate that old session. |

## Best short project summary

My project is a concurrent C banking server using TCP sockets and pthreads. The server owns binary data files and enforces role-based permissions for customers, employees, managers, and administrators. It protects records using fcntl locks, serializes balance changes with a named semaphore, uses fsync and recovery journals for durable updates, and records all customer money movements in a passbook-style transaction file.